

HelixSpecifier

HelixSpecifier

Развој вођен спецификацијама који сам прилагођава церемонијал обиму посла.

Сажетак

HelixSpecifier је Go мотор који спаја три методологије развоја — ток рада вођен спецификацијама из SpecKit-а, дисциплину TDD-а из Superpowers-а и животни циклус прекретница из GSD-а — у један прилагодљиви процес. Сваки задатак класификује према потребном труду и прилагођава количину процеса према томе.

Кратак опис

HelixSpecifier је мотор за развој вођен спецификацијама намењен AI агентима. Комбинује SpecKit, Superpowers и GSD, класификује посао према нивоу труда, спроводи фазе спецификације уз подршку дебате, намеће минимални однос тестова и кода, и учи из сваког завршеног тока.

Детаљан опис

HelixSpecifier је мотор за развој вођен спецификацијама (SDD) написан у Go (модул `digital.vasic.helixspecifier`), изграђен као компонента HelixAgent ансамбла AI агената. Спаја три праксе развоја које обично постоје у три одвојена алата — и три одвојена начина размишљања — у један прилагодљиви ток рада: седмофазни SDD процес из SpecKit-а (Constitution, Спецификација, Разјашњење, Планирање, Задаци, Анализа, Имплементација), TDD дисциплину из Superpowers-а са паралелним извршавањем подагента, и управљање животним

циклом прекретница из GSD-а. Сваки стуб наставља да ради оно у чему је најбољи; мотор је оно што их повезује у јединствен кохерентан ток уместо ручно спајаног процеса.

Његова централна идеја је *прилагодљиви церемонијал*: мотор класификује долазне задатке према нивоу труда и прилагођава количину процеса, тако да једнолинијска исправка не пролази кроз исти захтевни ритуал као велика функционалност — а велика функционалност се не пропушта са ригорозношћу намењеном типографској грешци. На тој основи изграђено је десет кључних функционалности: паралелно извршавање задатака са ограниченом конкурентношћу, машински читљиви „Constitution као Код” који аутоматски намеће обавезна правила, „Њуистов TDD” који прати и намеће минимални однос тестова и имплементације, вишеструка дебата са више агената за дорађивање спецификације, адаптивно учење нивоа вештина, анализа постојећег кода, предиктивна спецификација заснована на историјским обрасцима, пренос знања између пројеката, прилагођавање церемонијала у току извршавања и трајна меморија спецификација са семантичком претрагом.

Користи се као Go модул — путем `go get` или локалне `replace` директиве — иза намерно једноставног API мотора: региструју се три стуба плус скалер церемонијала и меморија спецификација, класификује се труд потребан за посао, затим се извршава цео ток и добија резултат са оценом квалитета. Површина је једноставна; оркестрација иза ње није. Као и остатак Helix породице, развија се под режимом верификације без блефирања, са тестовима који проверавају стваран код уместо симулација.

Зашто смо га изградили

Развој вођен спецификацијама, ригорозан TDD и управљање прекретницама обично су три одвојене праксе са три одвојена алата. HelixSpecifier је изграђен како би AI агент (HelixAgent) могао да извршава сва три као један кохерентан, самоприлагођавајући ток уместо да их ручно спаја.

Зашто је ово револуционарно

Аутоматски прилагођава процес обиму посла. Тимови обично западну у једну од две крајности: претерану церемонијалност у свему (безбедно, али споро и потајно омражено) или потпуни недостатак церемонијалности (брзо док не престане да буде). HelixSpecifier укида тај компромис тако што прилагођава церемонијалност класификованом напору за сваки задатак и поново је подешава током извршавања, како се посао открива.

Могућност која раније није била практична јесте процес који се сам прилагођава сваком задатку — а поврх тога, спецификационе одлуке подржане вишеструком, вишекорисничком расправом са оцењивањем ставова, уместо првог нагађања једног агента.

Шта је иновативно

- **Адаптивна церемонијалност** — ниво процеса вођен метрикама квалитета у реалном времену и прилагођаван током извршавања, а не унапред фиксиран.
- **Најквист TDD** — контрола односа тестова и имплементације (минимум 2х), заснована на логици Најквистове теореме о узорковању: да бисте верно ухватили понашање, морате га узорковати знатно изнад његове фреквенције, па тестови морају надмашити код који покривају.
- **Архитектура расправе** — вишестепено, вишекорисничко дорађивање спецификације, где се предлози износе, оцењују и конвергирају, замењујући појединачно мишљење противничким приступом.
- **Предиктивна спецификација и пренос знања између пројеката** — мотор анализира акумулиране токове како би предвидео спецификације и пренео стечено знање из једног пројекта у следећи.
- **Constitution као код** — обавезна правила пројекта претворена у машински читљиве и спровођена помоћу мотора, уместо да се ослањају на будност ревизора.

Највећи технички изазови и како смо их решили

- **Спајање три методологије без међусобног сукоба** — SpecKit, Superpowers и GSD свака претпоставља да има контролу над радним током. Решено фузионим мотором који региструје сваки стуб иза заједничког интерфејса и покреће их кроз један заједнички животни циклус тока, тако да се стапају у јединствен процес уместо да се сударају.
- **Одређивање колико процеса одређени задатак заправо захтева** — превише процене и све се вуче; премало и ризичан посао стиже на продукцију неконтролисан. Решено класификатором напора који процењује посао и напаја скалер церемонијалности који динамички прилагођава ниво процеса током извршавања.
- **Одржавање високог квалитета спецификације без људског контролора за сваку одлуку** — решено заменом једнократних спецификација дорађивањем

заснованим на расправи, где агенти оцењују конкурентске предлоге кроз рунде, и спровођењем Најквист TDD односа како имплементација не би претекла своје тестове.

Технолошки стек

- **Go** — изабран како би мотор био испоручиван као један увозни бинарни фајл без додатног терета током извршавања; његов модел конкурентности омогућава ограничено паралелно распоређивање задатака и вишестепене рунде расправе уместо проблема са нитима.
- **логрус** — структурирано логовање које се провлачи кроз мотор и сва три стуба, тако да су одлуке тока (класификација, промене церемонијалности, исходи расправе) читљиве и накнадно.
- **Стуб SpecKit** — седмофазни процес развоја вођен спецификацијом (Constitution → Спецификација → Разјашњење → Планирање → Задаци → Анализа → Имплементација), који пружа дисциплиновану основу за претварање спецификације у код.
- **Стуб Superpowers** — дисциплина TDD са паралелним извршавањем подагента, која обезбеђује ригорозност тестирања пре имплементације и паралелизацију која одржава искреност и брзину имплементације.
- **Стуб GSD** — управљање прекретницама и животним циклусом, дајући току осећај „завршености” и напредовања кроз фазе.
- **Меморија спецификација** — трајна, семантички претражива база прошлих спецификација, основа која омогућава предиктивну спецификацију и пренос знања између пројеката уместо да се сваки пут креће од нуле.

Напомене о статусу и искрености

- **Статус: бета.** Користи се као компонента модула Go унутар HelixAgent.
- **Лиценца: није одређена.** Није пронађена ЛИЦЕНЦА путем GitHub API — НЕВЕРИФИКОВАНО / није наведено.
- Приказано име “HelixSpecifier” упућује на репозиторијум specifier.

Приоритетни ниво: Helix-примарни.